

Universidad Nacional De Colombia

Facultad de minas

Programación Orientada a Objetos

Actividad 4: Valor 20%

Junio 2026

Docente

Walter Hugo Arboleda

Estudiante

Iván Camilo Barragán Echavarría

ibarragan@unal.edu.co



Enlace repositorio: <https://github.com/AtodeKimeru/hello-java/tree/main/Actividad4/src/main/java>

Ejercicio No. 1 - Página 400

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

"""

```
mvn compile exec:java -Dexec.mainClass="Excepciones.EjercicioNo1" -Dexec.classpath="Actividad4/src/main/java"
```

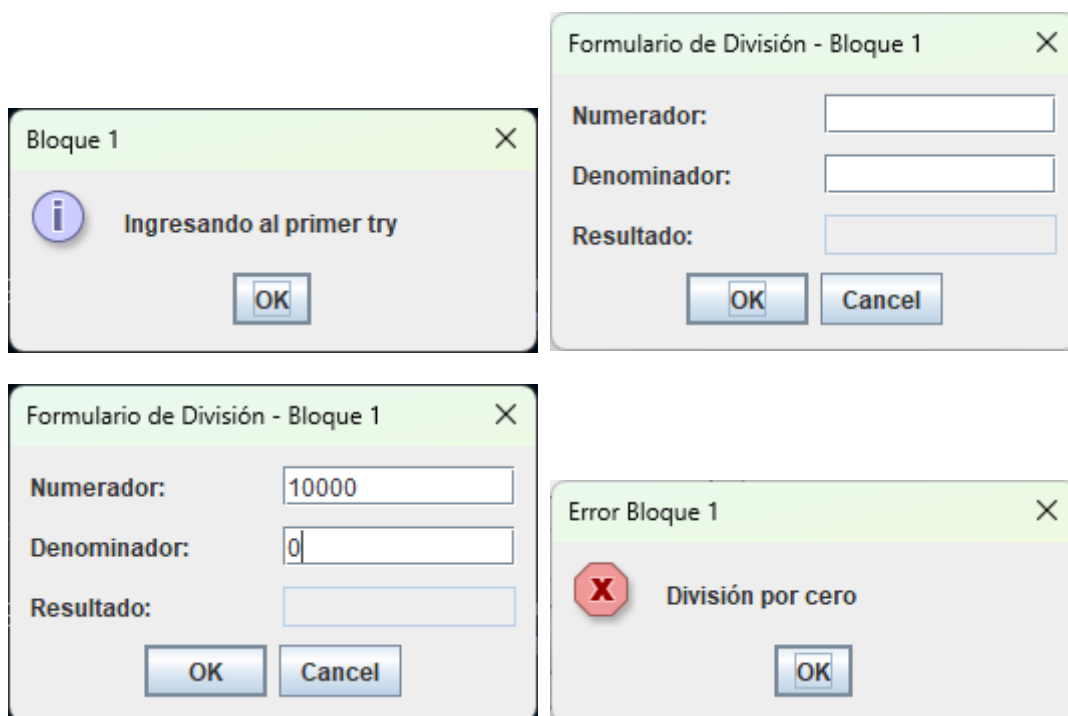
"""

Asegura estar en el directorio del proyecto Actividad4/

Ejemplo:

```
ATODEKIMERU@PCas MINGW64 ~/Documents/UN/P00/Actividad4 (main)
$ mvn compile exec:java -Dexec.mainClass="Excepciones.EjercicioNo1" -Dexec.classpath="Actividad3/src/main/java" -e
[INFO] Error stacktraces are turned on.
[INFO] Scanning for projects...
[INFO]
[INFO] -----< io.github.AtodeKimeru:actividad4 >-----
[INFO] Building actividad4 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO]
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ actividad4 ---
[INFO] skip non existing resourceDirectory C:\Users\ivanc\Documents\UN\P00\Actividad4\src\main\resources
[INFO]
Warning: GitGuardian - Please authenticate
```

Interfaz Gráfica:



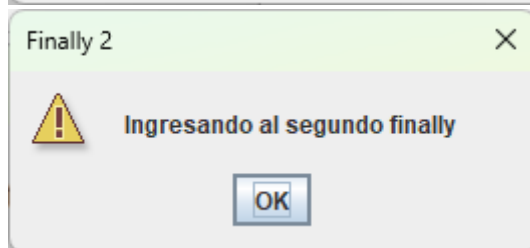
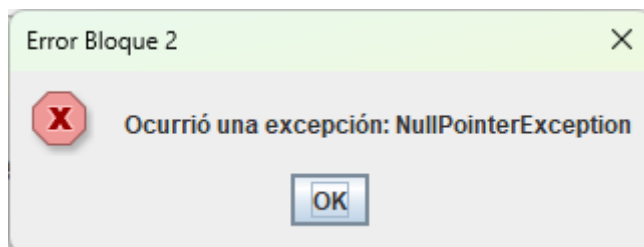
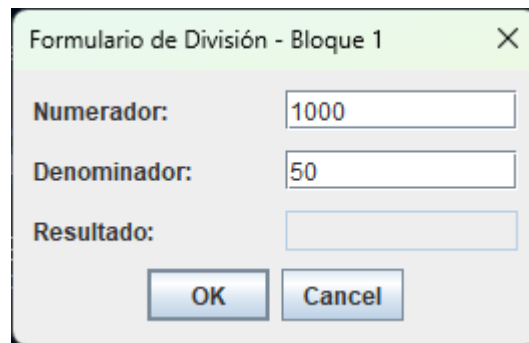
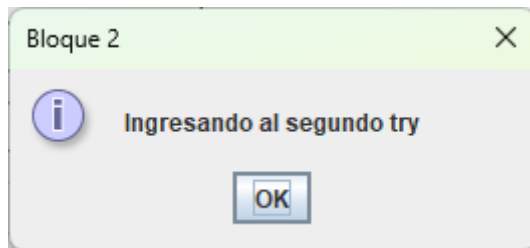
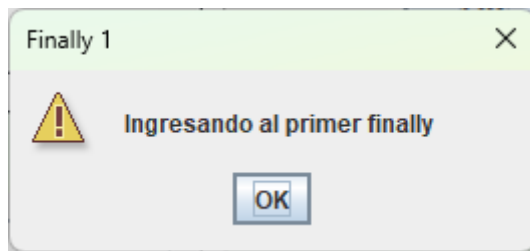
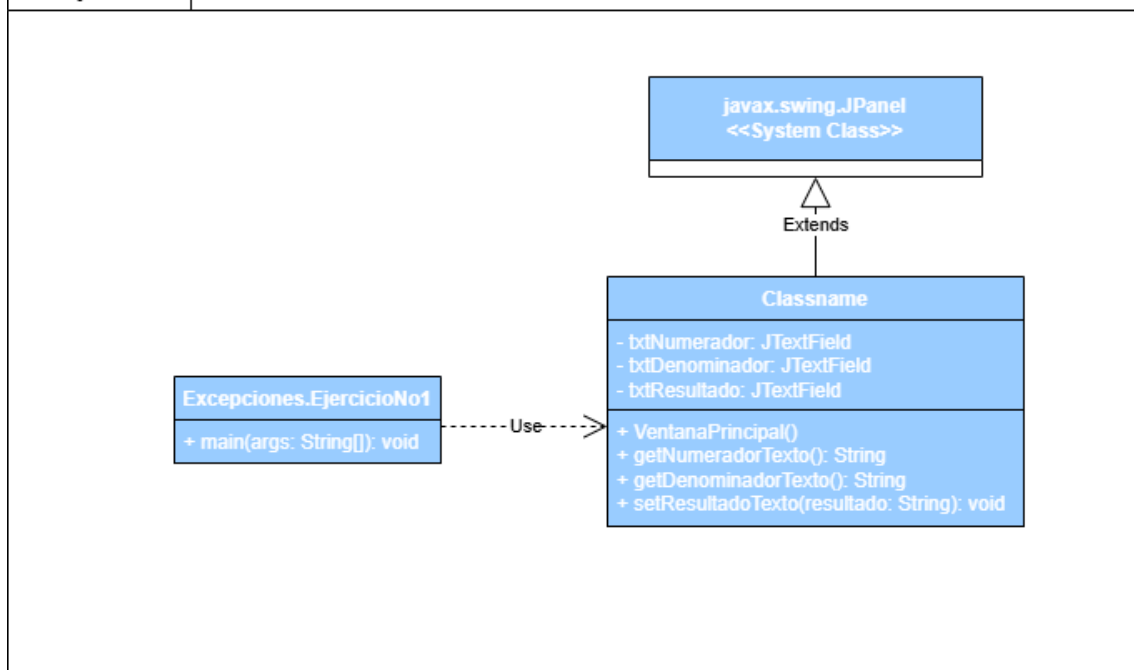


Diagrama de Clases:

Excepciones



Caso de uso:

1. Manejo de Excepción División por 0 (Bloque 1)

- **Entrada de Datos:** El programa presenta una ventana única con tres campos. Le pide al usuario un numerador y un denominador.
- **Funcionamiento (Sin errores):** Si el usuario ingresa números válidos (por ejemplo, 10000 y 2), el programa realiza la división matemática elemental ($\frac{10000}{2} = 5000$), muestra el resultado directamente en la pantalla y continúa de forma natural.
- **La Excepción Provocada:** Si el usuario ingresa un 0 en el denominador, el código activa una alerta de seguridad manual (throw new ArithmeticException). Esto se hace porque en Java, la división de decimales (double) con cero genera "Infinito" en lugar de colapsar. El programa interrumpe deliberadamente el cálculo normal para simular un error matemático crítico, saltando de inmediato al mensaje visual de "División por cero".
- **Garantía del Cierre:** Sin importar si la división fue exitosa, si el usuario escribió letras (provocando un NumberFormatException) o si el programa falló por el cero, el bloque finally se ejecuta obligatoriamente, mostrando el mensaje "Ingresando al primer finally".

2. Simulación de Excepción por error en el código (Bloque 2)

Una vez que termina el primer bloque, el programa avanza automáticamente a un escenario completamente distinto para demostrar la **especificidad de los errores**:

- **El Error Automático:** El programa intenta ejecutar una acción sobre un objeto inexistente (objeto.toString() donde objeto vale null). Esto provoca que la Máquina

Virtual de Java (JVM) colapse internamente y arroje de forma automática un error de sistema llamado `NullPointerException`.

- **Filtrado de Captura:** El programa tiene dos bloques `Catch`. El primero busca un `ArithmeticException` (el error del Bloque 1); como este error no aparece, lo deja pasar. El segundo busca un error genérico (`Exception e`), que atrapa cualquier excepción adicional e informa al usuario que tipo de fallo del sistema ocurrió.
- **Cierre del Ciclo:** Al igual que en el escenario anterior, pase lo que pase, el programa ejecuta el mensaje *"Ingresando al segundo finally"*.

Ejercicio No. 2 - Página 406

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

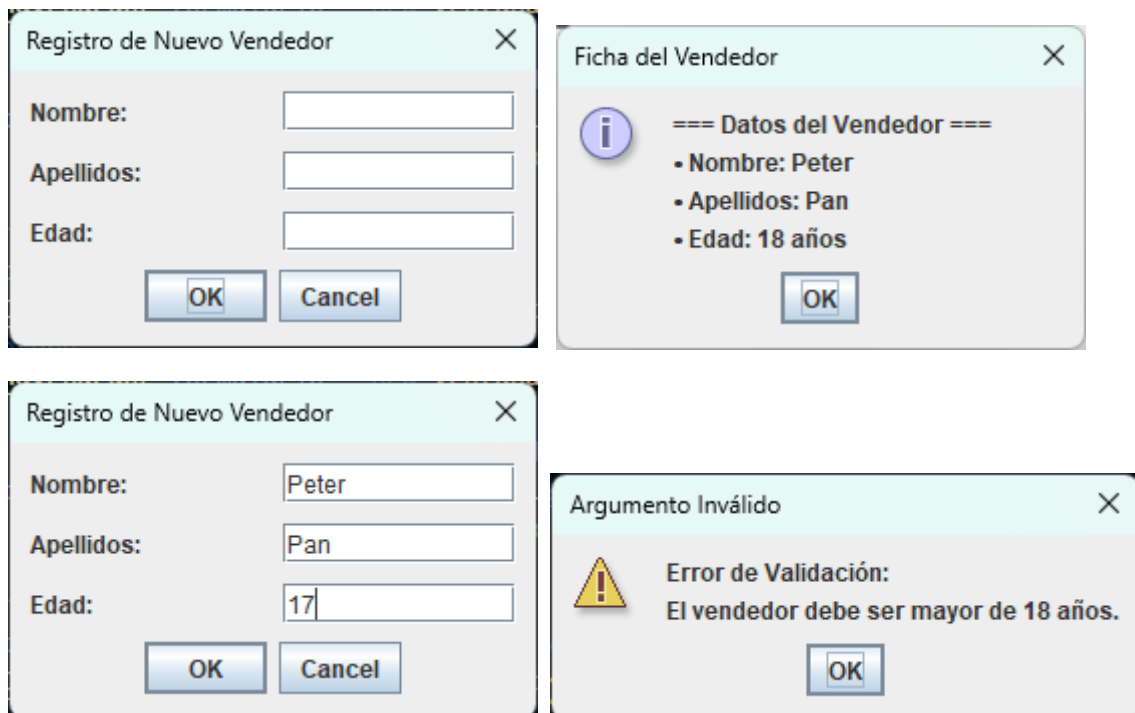
"""

```
mvn compile exec:java -Dexec.mainClass="Excepciones.EjercicioNo2" -  
Dexec.classpath="Actividad4/src/main/java"
```

"""

Asegura estar en el directorio del proyecto `Actividad4/`

Interfaz Gráfica:



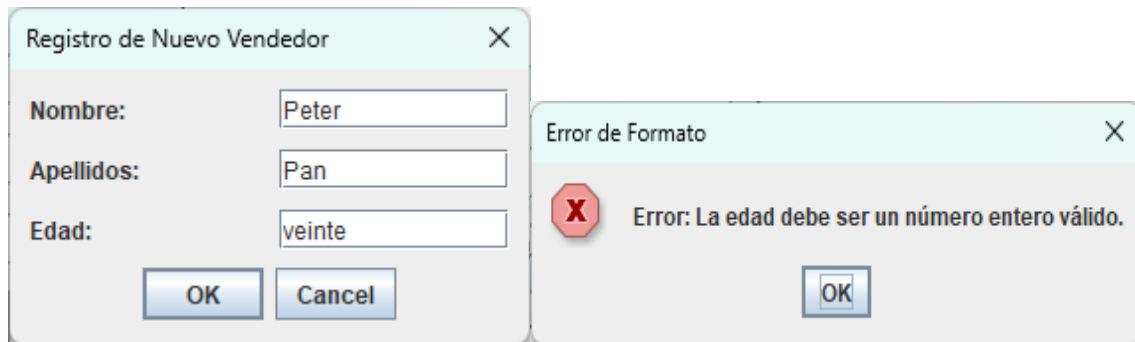
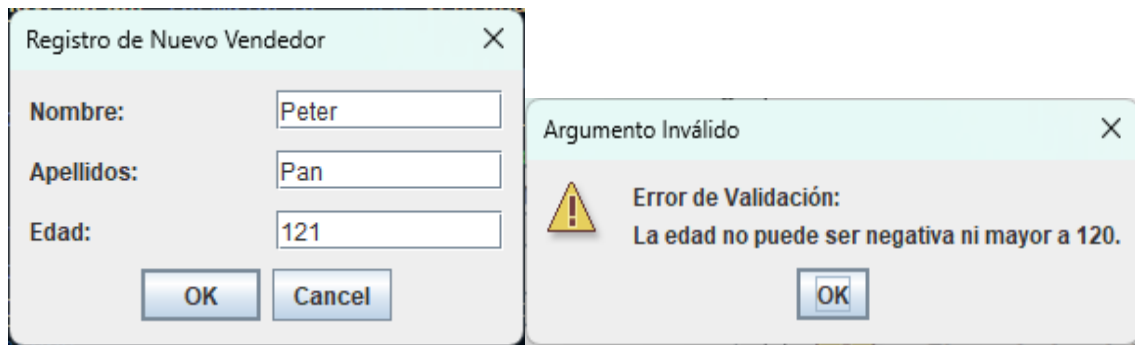
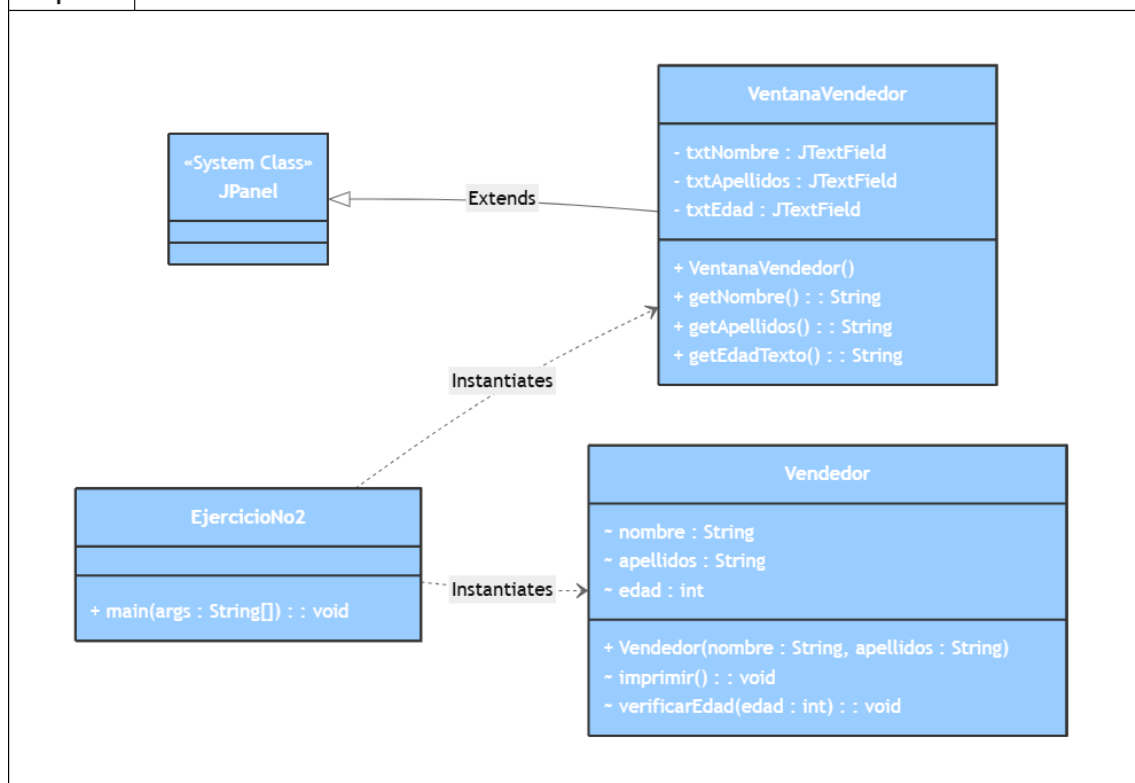


Diagrama de Clases:

Excepciones



Caso de uso:

1. Inicialización y Despliegue

- El flujo inicia en la clase (**EjercicioNo2**) al arrancar el programa.
- El controlador instancia la interfaz gráfica (**VentanaVendedor**) y la monta dentro de un panel unificado para solicitar los datos del usuario en una sola pantalla.

2. Captura y Filtrado de Datos

- El usuario interactúa con la ventana única, digita el *Nombre*, *Apellidos* y *Edad*, y presiona el botón "Aceptar" (OK).
- **Control de flujo:** Si el usuario decide cerrar la ventana o presionar "Cancelar", el controlador detecta la acción y detiene el proceso de forma segura enviando un mensaje de cancelación.
- Si el usuario continúa, el controlador extrae las cadenas de texto del formulario y realiza la primera validación técnica: verifica que la edad ingresada sea un número entero válido. Si el usuario ingresó letras (ej. *"veinte"*), el sistema atrapa un `NumberFormatException` y muestra una alerta gráfica de error sin tumbar la aplicación.

3. Validación de Reglas de Negocio

El controlador (**EjercicioNo2**) crea el objeto **Vendedor** y le envía la edad para que el propio modelo aplique sus restricciones lógicas a través del método `verificarEdad(edad)`:

- **Filtro Biológico:** Si la edad es menor a 0 o mayor a 120 años, el modelo frena la operación lanzando un `IllegalArgumentException` con el mensaje: *"La edad no puede ser negativa ni mayor a 120"*.
- **Filtro Laboral Legal:** Si la edad está en el rango correcto, pero es menor a 18 años, el modelo detiene la operación lanzando un `IllegalArgumentException` con el mensaje: *"El vendedor debe ser mayor de 18 años"*.

4. Finalización y Confirmación Exitosa

- Si la edad cumple rigurosamente ambas condiciones del modelo, el objeto **Vendedor** asigna el valor a su atributo de forma permanente.
- El controlador toma el control final e invoca al método `imprimir()` del vendedor, el cual despliega una ventana emergente limpia y estilizada de confirmación ("Ficha del Vendedor") mostrando el éxito del registro con todos sus datos consolidados.

Ejercicio No. 3 - Página 412

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

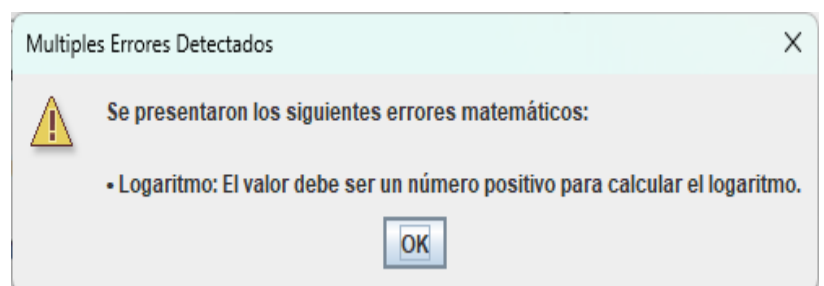
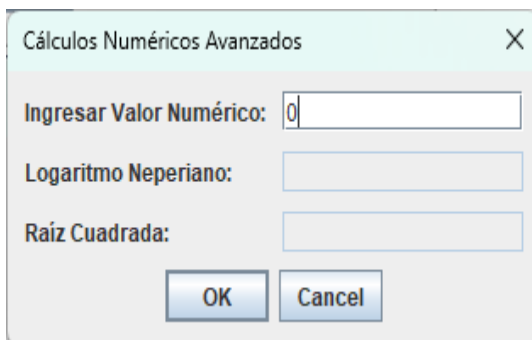
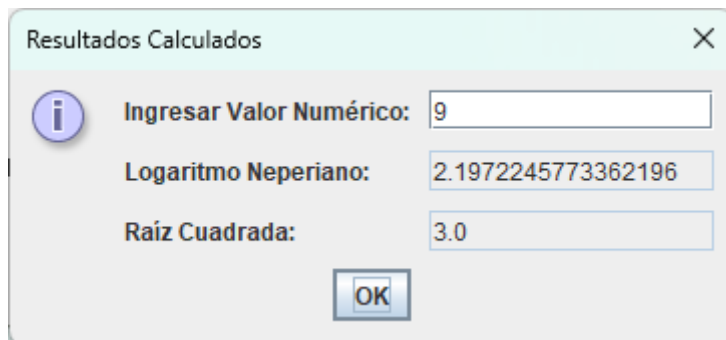
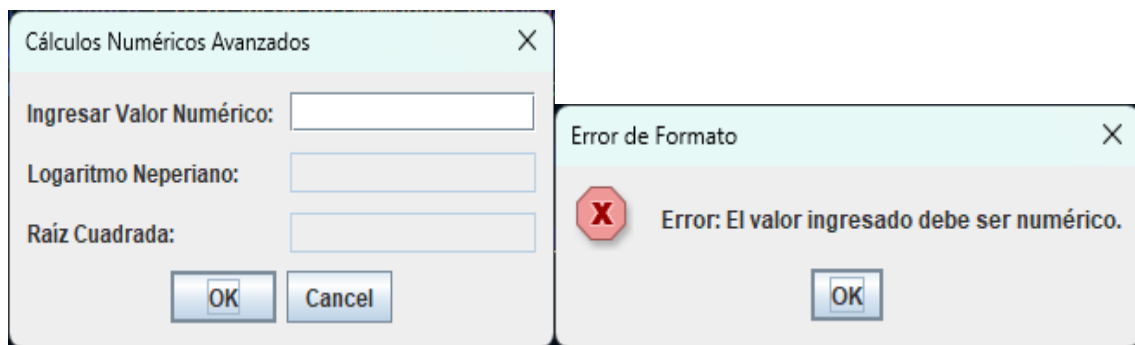
"""

```
mvn compile exec:java -Dexec.mainClass="Excepciones.EjercicioNo3" -  
Dexec.classpath="Actividad4/src/main/java"
```

"""

Asegura estar en el directorio del proyecto Actividad4/

Interfaz Gráfica:



Cálculos Numéricos Avanzados

Ingresar Valor Numérico:

Logaritmo Neperiano:

Raíz Cuadrada:

OK

Cancel

Multiples Errores Detectados

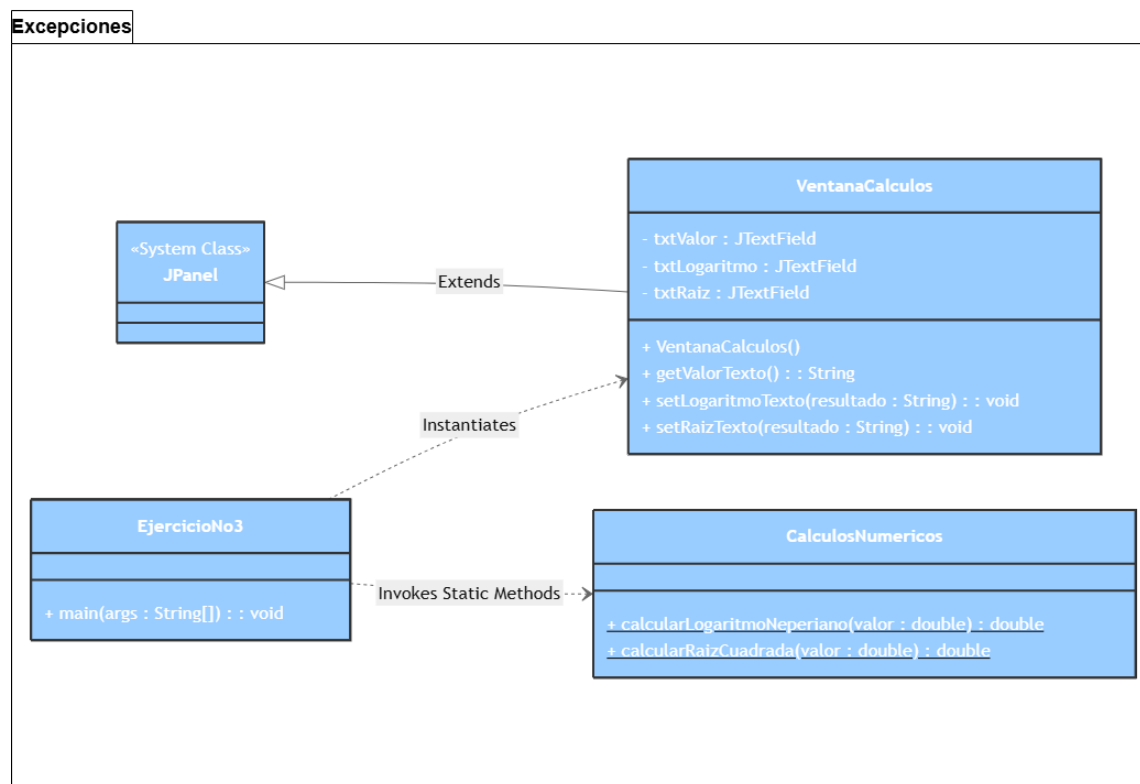


Se presentaron los siguientes errores matemáticos:

- Logaritmo: El valor debe ser un número positivo para calcular el logaritmo.
- Raíz Cuadrada: El valor debe ser cero o un número positivo para calcular la raíz cuadrada.

OK

Diagrama de Clases:



Caso de uso:

1. **Despliegue Coordinado:** El usuario arranca el sistema ejecutando EjercicioNo3. El controlador instancia (**VentanaCalculos**) y abre la ventana integrada pidiendo el valor numérico.
2. **Validación Sintáctica:** Al presionar "Aceptar", el controlador intercepta la cadena de caracteres. Si tiene un formato ilegal (letras), se interrumpe el flujo, se captura un `NumberFormatException` y se le muestra una advertencia gráfica al usuario.
3. **Validación Semántica Matemática:** Si es un número real válido, el controlador llama de forma estática al modelo `CalculosNumericos`. Si el número es negativo (ej. -25), el modelo frena la ejecución lanzando un `ArithmeticException` con su mensaje específico. El controlador lo atrapa de inmediato y lo dibuja en una ventana de error.
4. **Resultados Simultáneos:** Si el número es positivo (ej. 4), se calculan con éxito ambas operaciones de forma secuencial. El controlador escribe los resultados reales directamente en las cajas bloqueadas de la interfaz y muestra el formulario completo como confirmación del éxito del proceso.

Ejercicio No. 4 - Página 418

Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

```
"""
```

```
mvn compile exec:java -Dexec.mainClass="Excepciones.EjercicioNo4" -  
Dexec.classpath="Actividad4/src/main/java"
```

```
"""
```

Interfaz Gráfica:

Registro de Equipo

Nombre del Equipo:

Universidad:

Lenguaje Objetivo:

Error

 Todos los datos del equipo son obligatorios.

Datos del Integrante #1

Nombre del Integrante:

Apellidos del Integrante:

Error de Validación

 Errores detectados en la información:

- Nombre: El campo no puede estar vacío.
- Apellidos: El campo no puede estar vacío.

Error de Validación

 Errores detectados en la información:

- Nombre: El nombre/apellido no puede tener dígitos numéricos.
- Apellidos: El nombre/apellido no puede tener dígitos numéricos.

Error de Validación

 Errores detectados en la información:

- Nombre: La longitud no debe ser superior a 20 caracteres.

Error de Validación

 Errores detectados en la información:

- Apellidos: El campo no puede estar vacío.

OK

Message

 Integrante registrado con éxito.

OK

Registro Incompleto

 Un equipo requiere un mínimo de 2 integrantes para competir.

OK

Datos del Integrante #2

Nombre del Integrante:

Apellidos del Integrante:

OK Cancel

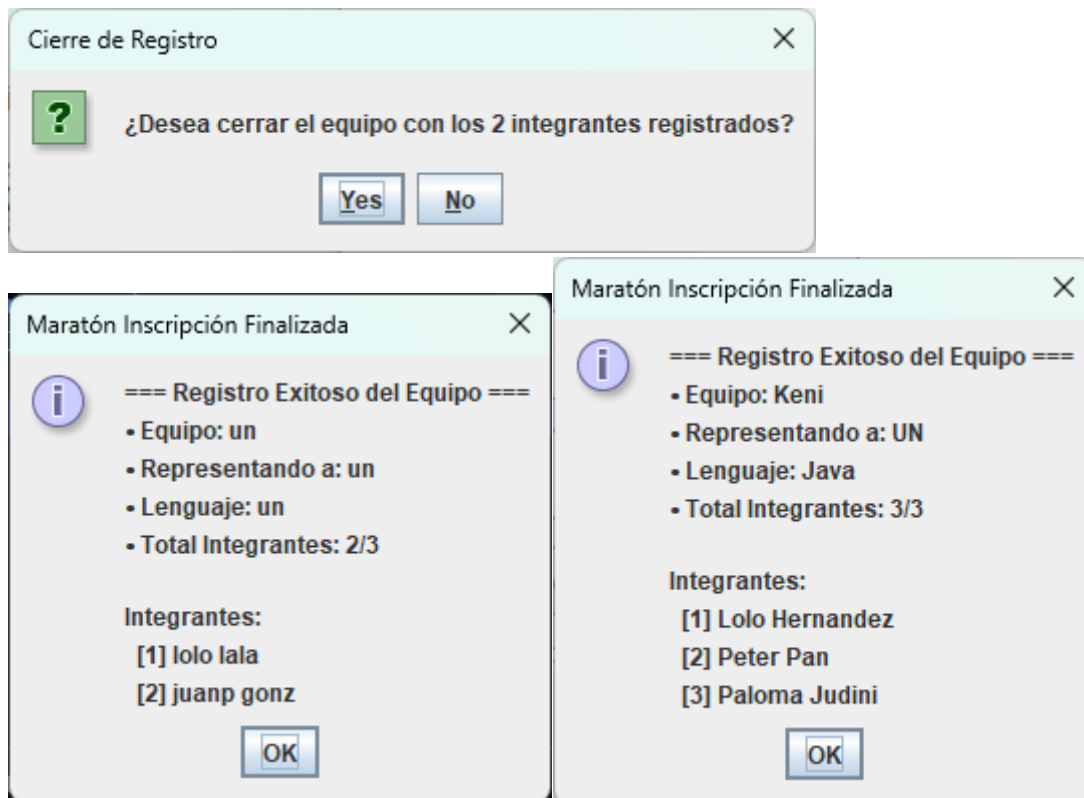
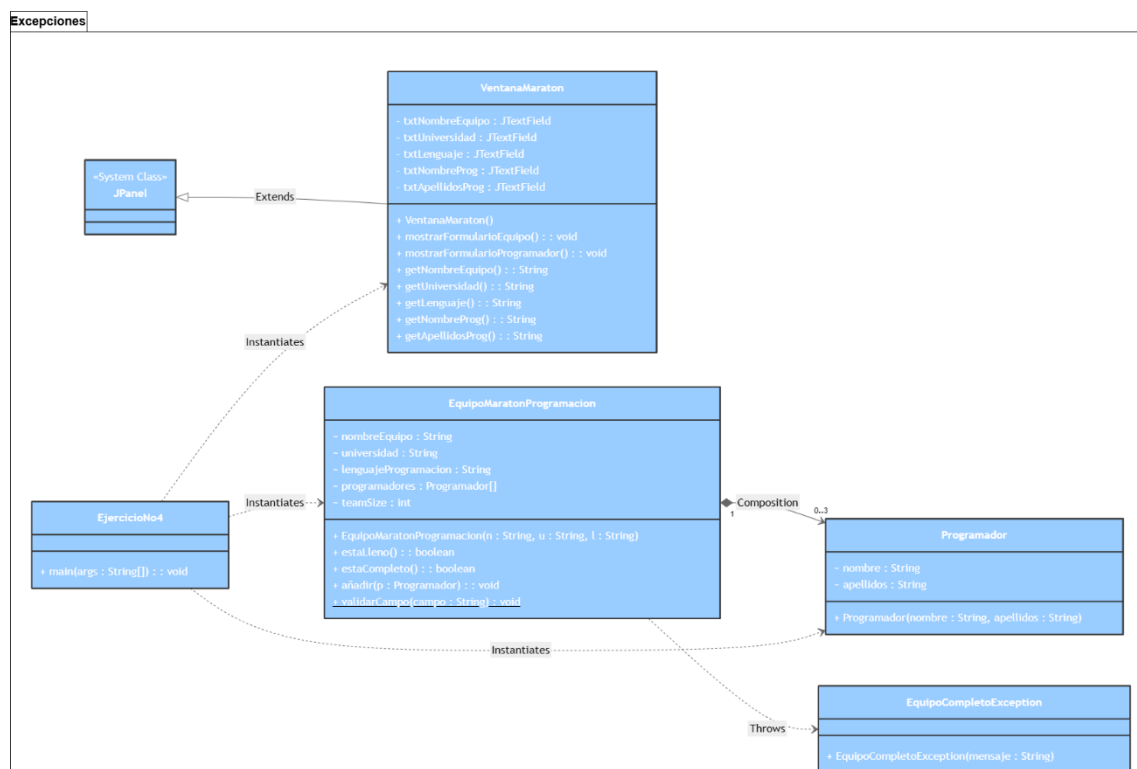


Diagrama de Clases:



Caso de uso:

1. **Estructuración del Perfil:** El controlador arranca la (**VentanaMaraton**) en su fase inicial. El usuario registra la base del equipo (*Nombre, Universidad, Lenguaje*)

2. **Ciclo de Carga Iterativa:** El controlador analiza las cuotas de capacidad del modelo (**EquipoMaratonProgramacion**). Mientras no se alcance el límite máximo (3 programadores), la interfaz muta usando su diseño interno dinámico y solicita los nombres y apellidos de cada programador de manera secuencial.
3. **Filtros Sintácticos Estrictos:** Al confirmar la carga de un integrante, el controlador invoca las reglas estáticas del modelo para limpiar los nombres. Si un campo contiene números o excede los 20 caracteres reglamentarios, se recolectan simultáneamente los desbordes mediante aislamiento controlado y se alerta gráficamente al usuario, repitiendo el paso actual sin perder el progreso global del equipo.

Validación de Reglas de Negocio:

4. Si el usuario intenta forzar la salida teniendo solo un integrante, el controlador bloquea la acción informando que incumple las bases de la competencia (mínimo 2 integrantes).
5. Si el usuario intenta agregar un cuarto integrante burlando los bucles, el modelo arroja una excepción explícita de (**EquipoCompletoException**) interrumpiendo la inserción.
6. **Cierre de Matrícula:** Al llegar al estado óptimo o tras un cierre voluntario válido (2 o 3 integrantes), el controlador recopila las entidades del modelo y genera un informe de inscripción gráfico detallado consolidando el éxito del caso de uso.

Ejercicio No. 5 - Página 427

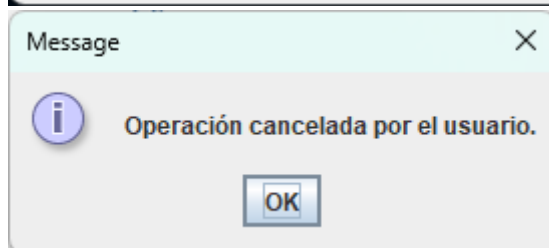
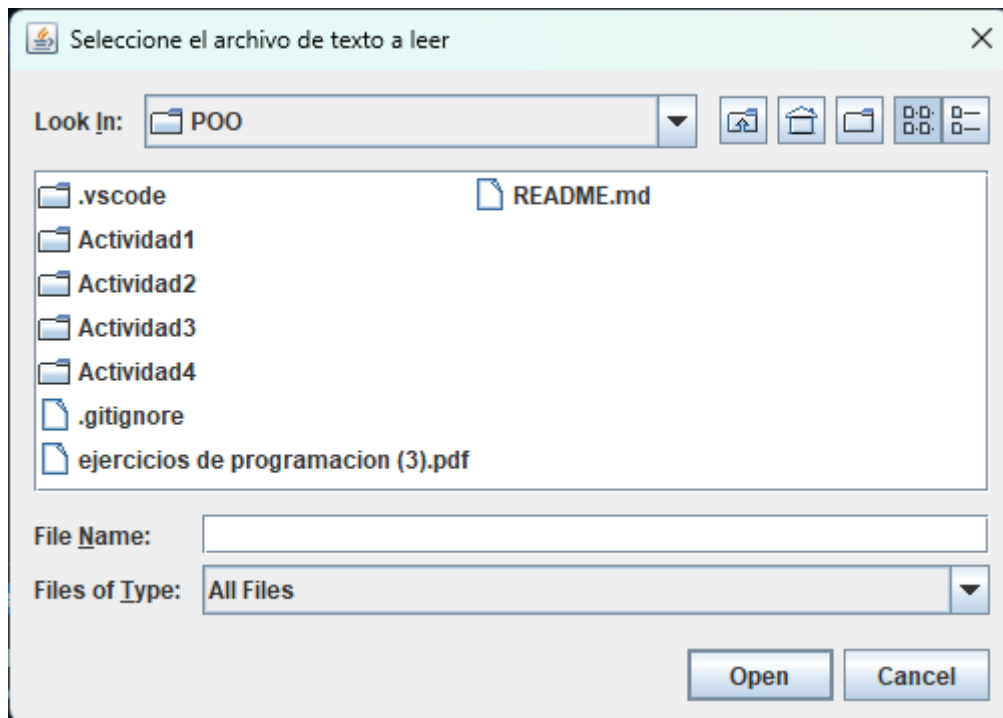
Para compilar este ejercicio usando Maven, se debe correr en terminal la siguiente línea de código:

```
"""
```

```
mvn compile exec:java -Dexec.mainClass="Archivos.EjercicioNo5" -  
Dexec.classpath="Actividad4/src/main/java"
```

```
"""
```

Interfaz Gráfica:



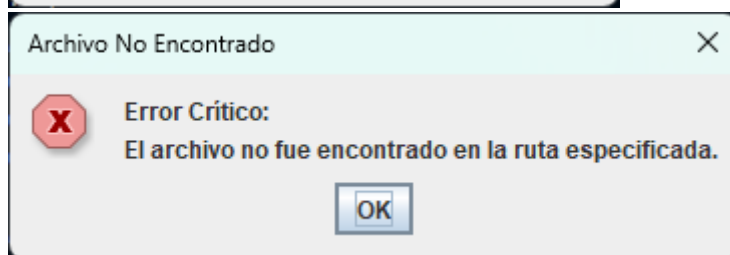
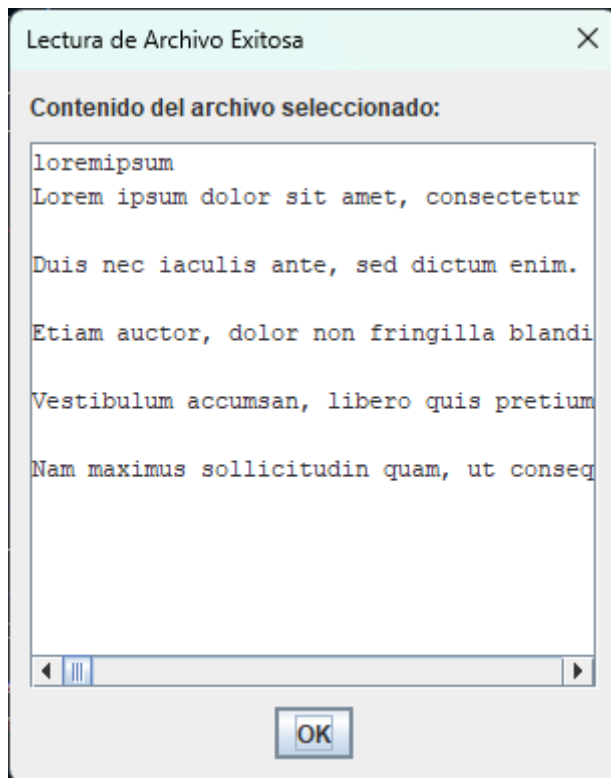
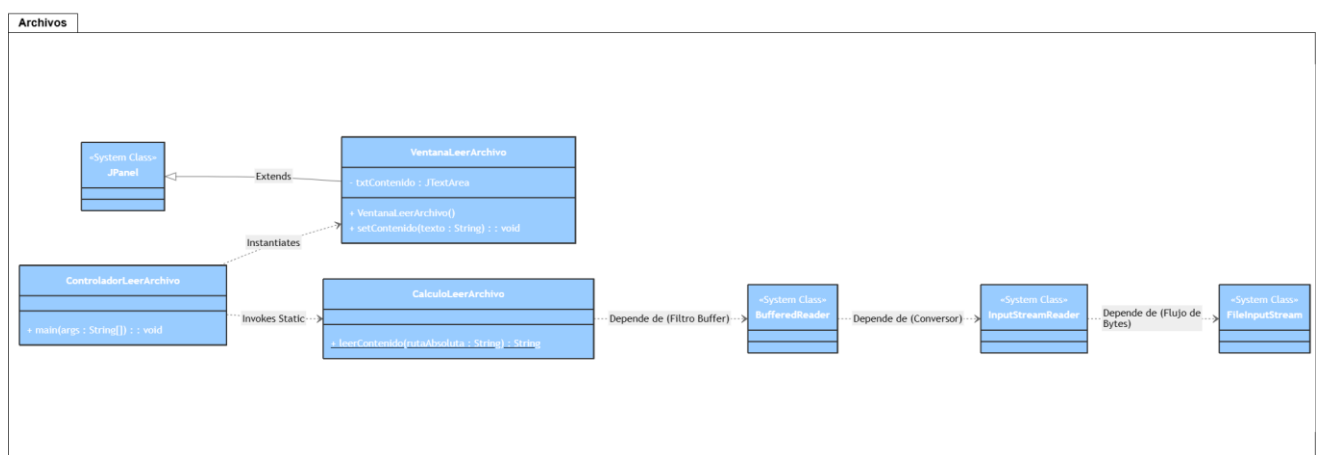


Diagrama de Clases:



Caso de uso:

1. **Disparador e Interacción Inicial:** El controlador inicia la ejecución en (**EjercicioNo5**). En lugar de forzar una ruta estática en código, levanta de forma asíncrona un componente gráfico del sistema operativo (JFileChooser) para que el usuario busque visualmente su archivo en el disco.

2. **Abstracción del Archivo:** Cuando el usuario elige el archivo .txt y confirma, el controlador captura la cadena con la dirección física exacta del archivo y llama al método estático del modelo (**CalculoLeerArchivo**).
3. **Procesamiento de la Cadena de Dependencias:** El modelo activa el motor de I/O de Java:
 - Abre un canal binario directo (FileInputStream).
 - Como los bytes crudos no se pueden leer como texto plano, inyecta ese canal en un decodificador de caracteres (InputStreamReader).
 - Para optimizar el rendimiento y no leer letra por letra (lo cual ralentizaría la computadora), envuelve el decodificador en un buffer de memoria (BufferedReader) para extraer líneas completas mediante readLine().
4. **Tratamiento de Excepciones de Infraestructura:**
 - Si el archivo fue borrado o movido justo antes de abrirlo, el modelo frena la operación arrojando FileNotFoundException.
 - Si el archivo tiene un bloqueo de permisos del sistema operativo, arroja IOException.
 - En cualquiera de estos casos, el controlador captura el error y le muestra una ventana emergente roja de advertencia al usuario.
5. **Aseguramiento de Cierre y Despliegue:** Gracias al bloque finally, pase lo que pase (lectura exitosa o interrupción por error), el programa cierra de forma obligatoria los flujos abiertos para no dejar la memoria del sistema operativo saturada. Si todo fue exitoso, los datos acumulados se inyectan en el área de texto de (**VentanaLeerArchivo**) y se muestran en una ventana flotante limpia.